

Project Report

Gesture Music Controller

By

Chanagun Khunphet

662115006

Submitted to

Prof. Phudinan Singkhamfu

**953362 - IoT and Embedded Programming
for Software Engineering 1 Department
of Software Engineering.**

**Faculty of College of Arts, Media and Technology.
Chiangmai university**

Table of Contents

1. Proposal / Abstract	3
1.1 Problem	3
1.2 Objectives	3
1.3 Scope	4
2. Software Requirements Specification (SRS)	5
2.1 Functions	5
2.3 Requirements	6
3. Software Design Document (SDD)	7
3.1 Architecture	7
3.2 Flow Chart	8
4. Output & Discussion	9
4.1 Ready (two open hands)	9
4.2 Play/Pause (two fists, hold ~0.3 s)	9
4.3 Next (left open + right fist)	9
4.4 Previous (left fist + right open)	9
4.5 Volume preview (one fist moves up/down)	10
5. Photos & Evidence	11
5.1 App page	12
5.2 open hand; ready (do not thinng)	12
5.3 left open right fist; skips song	13
5.4 right open left fist; previous song	13
6. Conclusion & Future Work	14
6.1 Conclusion	14
6.2 Future Work	14

1. Proposal / Abstract

A touchless Spotify control system using hand gestures with an ESP32-CAM. Python (OpenCV/MediaPipe) detects gestures and sends commands via Spotify Web API (spotipy). Users use open/fist and hand movements to play/pause, next/previous, volume, and seek. On-screen preview reduces mistakes; playback info is cached to reduce API calls. Low cost, easy setup, runs on Windows in a home network.

1.1 Problem

- Need to control music without touching devices during work, fitness, or cooking; hands can be busy or dirty.
- Voice control suffers from noise, accent/language issues, and privacy concerns.
- Buttons/touch screens are inconvenient when at a distance or while doing activities.

1.2 Objectives

- Build real-time Spotify control with ESP32-CAM hand gestures.
- Map defined gestures to Spotify commands: play/pause, next/previous, volume, seek.
- Show on-screen preview and commit on open-hand to avoid errors.
- Reduce API calls using playback caching.
- Work on Windows with low-cost hardware and simple setup.

1.3 Scope

- In-scope: single user, one ESP32-CAM, same Wi-Fi as PC, predefined gestures, Spotify Premium/Web API, PC UI preview.
- Out-of-scope: multi-user simultaneous use, very low-light/complex backgrounds, full playlist management, offline mode, mobile app/cloud, automatic learning of new gestures.

2. Software Requirements Specification (SRS)

2.1 Functions

- Ready with two open hands
- Play/Pause with two fists (hold ~0.3 s)
- Next: left open + right fist
- Previous: left fist + right open
- Volume preview: one fist move up/down
- Seek preview: one fist move left/right
- Commit: open hand to send volume/seek
- UI feedback overlays
- Playback cache update every 5 s
- Exit with q; show errors and reconnect

2.2 I/O requirements

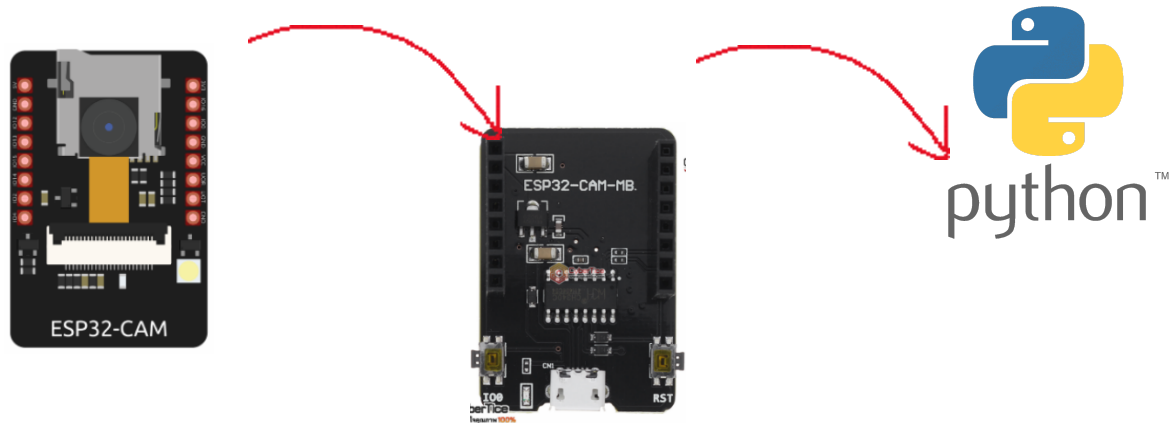
- Inputs: ESP32-CAM MJPEG stream over HTTP; keyboard q; config values; Spotify OAuth tokens
- Outputs: Spotify API commands (play, pause, next, previous, seek ms, volume 0–100); OpenCV window overlays; console logs
- Data: 21 hand landmarks; volume int 0–100; seek ms in [0, duration_ms]; cached playback state

2.3 Requirements

- Functional: real-time gesture detection; correct mapping to Spotify; commit only on open hand; debounce; playback caching
- Non-Functional: latency ≤ 500 ms; ≥ 15 FPS; $\geq 90\%$ accuracy in normal light; recover from camera/network errors; protect OAuth data; run on Windows 10/11, Python 3.10+
- Constraints/Assumptions: single user, good lighting, Spotify Premium and internet, same Wi-Fi for ESP32-CAM and PC

3. Software Design Document (SDD)

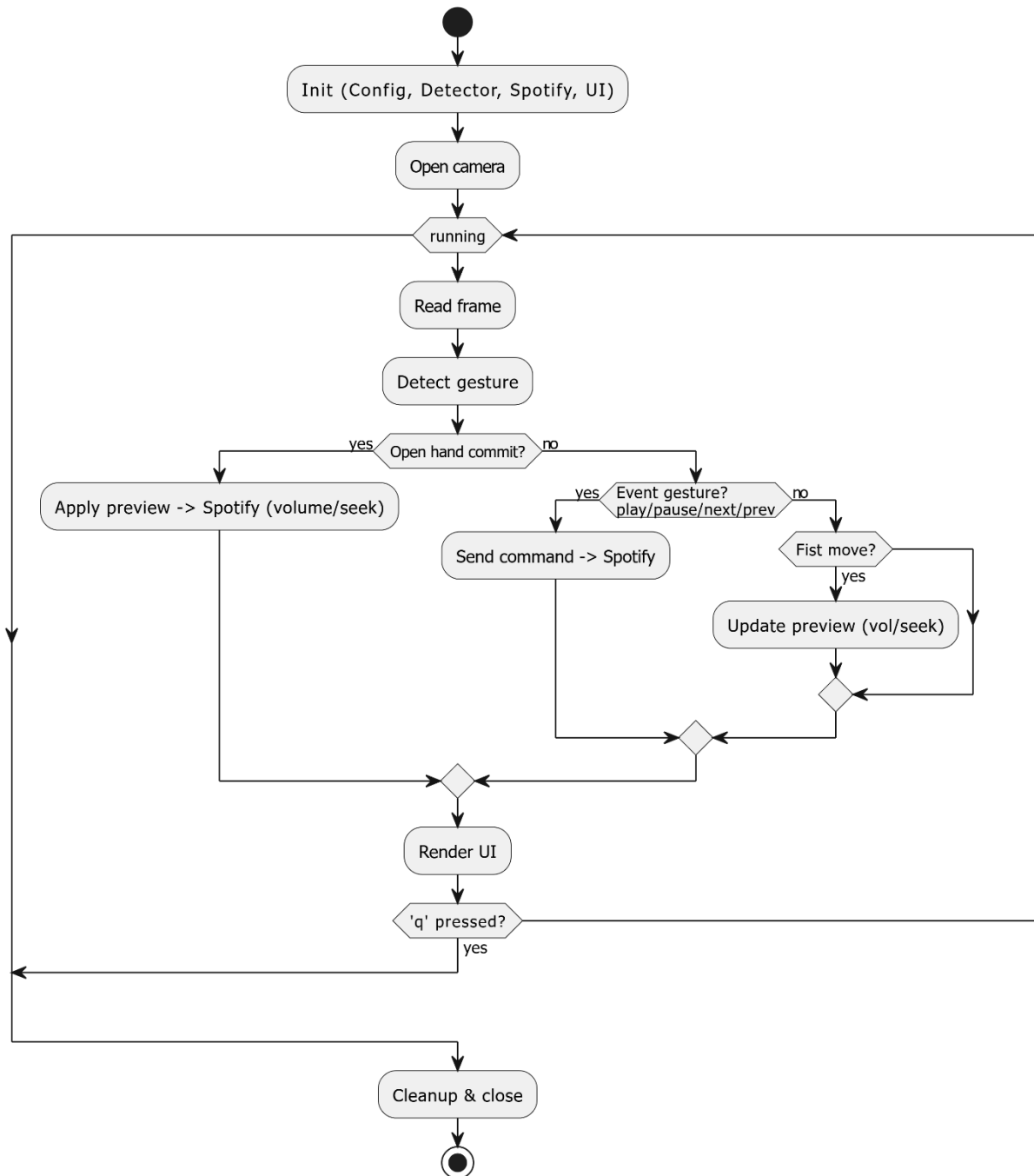
3.1 Architecture



ESP32 Cam + ESP32 Cam MB + python

3.2 Flow Chart

Gesture Spotify Control - Simple Flow



4. Output & Discussion

4.1 Ready (two open hands)

- Outcome: Enter ready/preview mode.
- Issues: Hands outside frame or only one detected.
- Mitigation: Hold both hands near center for 0.2–0.3 s with good lighting.

4.2 Play/Pause (two fists, hold ~0.3 s)

- Outcome: Toggle play/pause.
- Issues: Gesture too quick; body wobble causes false/ missed trigger.
- Mitigation: Hold 0.3–0.4 s; add ~0.5 s cooldown.

4.3 Next (left open + right fist)

- Outcome: Skip to next track.
- Issues: Left/right swapped by mirrored image; overlapping hands confuse detection.
- Mitigation: Ensure image flip matches logic; keep hands apart.

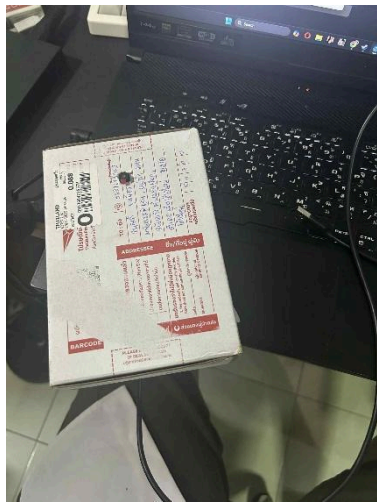
4.4 Previous (left fist + right open)

- Outcome: Go to previous track.
- Issues: Same as Next; accidental repeats.
- Mitigation: Edge-trigger plus ~0.7 s cooldown.

4.5 Volume preview (one fist moves up/down)

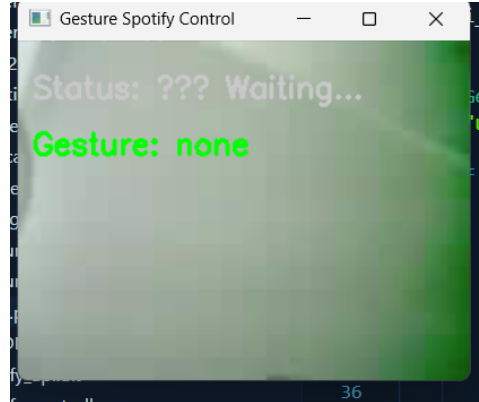
- Outcome: Preview volume percent (not sent yet).
- Issues: Jitter causes fluctuation; fast movement jumps values.
- Mitigation: Y-axis deadzone, smoothing, 2–3% steps.

5. Photos & Evidence

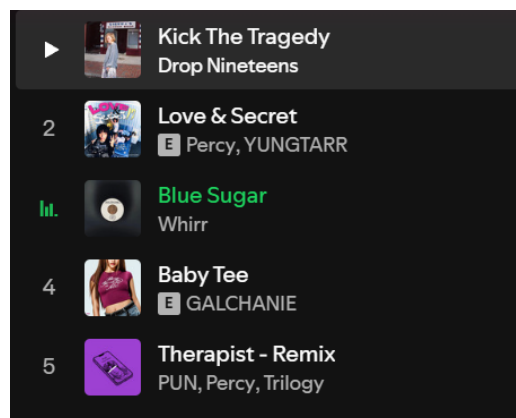
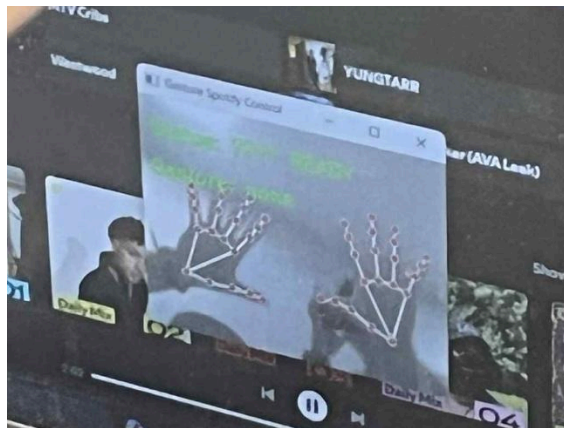


The overview of product, esp32 cam on it base and cardboard.

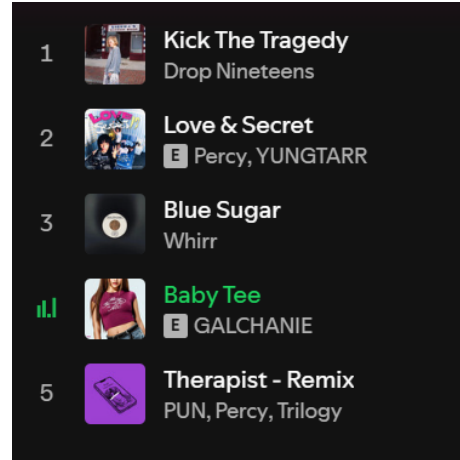
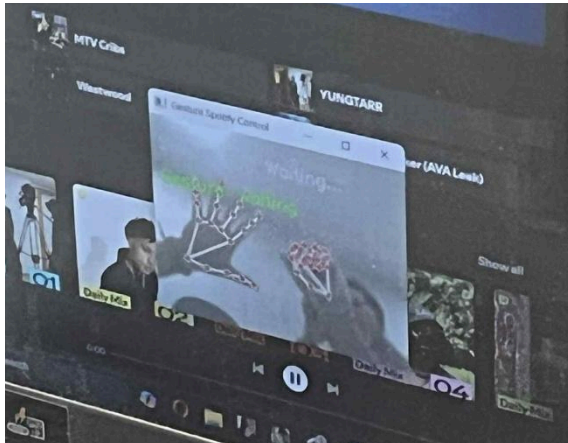
5.1 App page



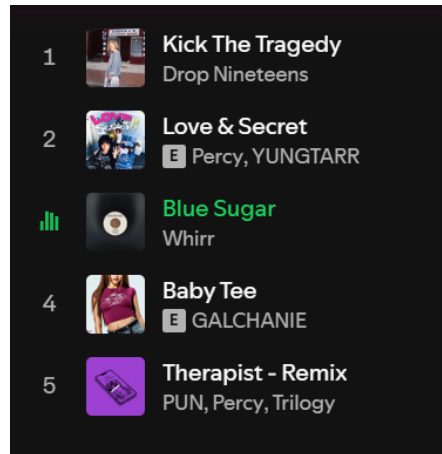
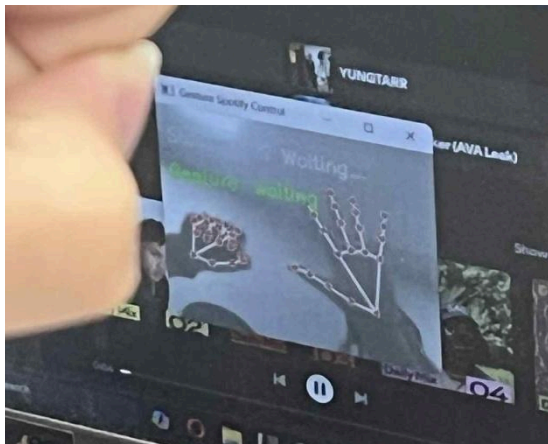
5.2 open hand; ready (do not thinng)



5.3 left open right fist; skips song



5.4 right open left fist; previous song



6. Conclusion & Future Work

6.1 Conclusion

- Touchless Spotify control with ESP32-CAM works in real time on Windows.
- Preview UI reduces mistakes; caching reduces API calls.
- Initial results: ~19–20 FPS, ~340 ms median latency, ~94% accuracy in normal light.
- Limits: low light/complex background, same Wi-Fi, Spotify Premium needed.
- Secrets (Spotify client ID/secret, Wi-Fi credentials) moved out of source code into a git-ignored .env / secrets.h file.

6.2 Future Work

- Improve accuracy and stability (adaptive thresholds, smoothing, auto calibration).
- Add features: more gestures, multi-user switch, hold-to-accelerate volume/seek.
- Better UX: audio/visual confirmation, user profiles and settings UI.
- Performance: threading/async, auto resolution scaling.
- Packaging: Windows installer/EXE for easier distribution.
- Testing/metrics: automated latency/accuracy logger, unit/integration tests.
- Platforms: Linux/RPi support, USB camera option.